

ForkJoin Framework

ForkJoin Framework

- Розміщує задачі в пул потоків `ForkJoinPool` extends `AbstractExecutorService`
- Забезпечує більшу швидкість за рахунок використання `work-stealing` алгоритму: потоки, які вільні, можуть здійснювати «крадіжку» роботи у потоків, які є зайнятими.
- Розроблений для алгоритмів, в яких підзадачі створюються рекурсивно
- Основні класи фреймворку

```
ForkJoinPool (implements ExecutorService),  
ForkJoinTask (implements Future), RecursiveTask  
(implements Future),  
RecursiveAction (implements Future)
```

- ! Алгоритми ForkJoin фреймворку використовуються класом `Arrays` для паралельного сортування об'єктів:
`Arrays.parallelSort()`
- ! Алгоритми ForkJoin фреймворку використовуються `java.util.streams` для паралельної обробки масивів

Етапи розробки паралельного алгоритму

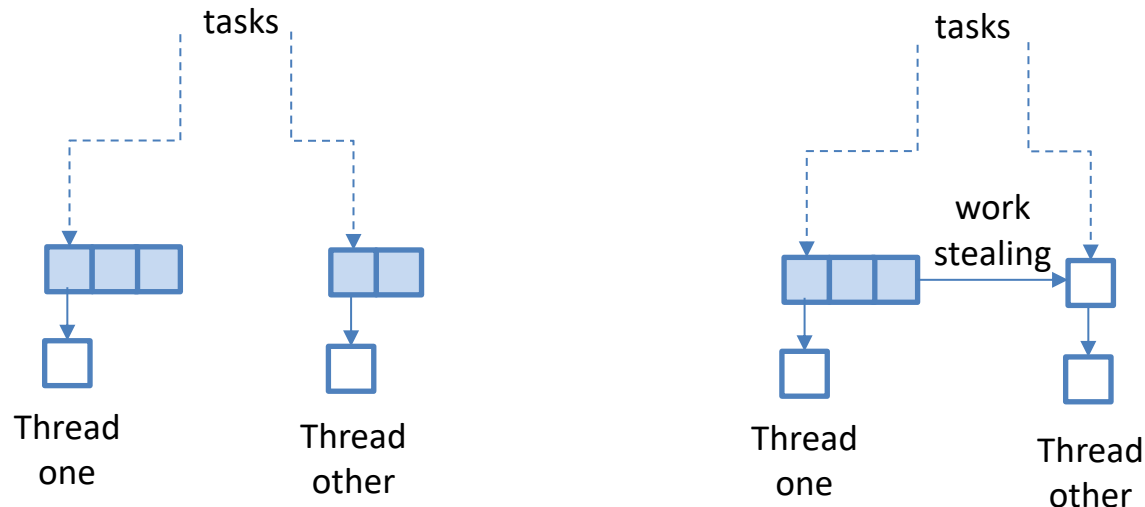
- Задача поділяється на підзадачі, які можуть виконуватись незалежно. Підзадачі можуть розроблятись як такі, що створюють нові підзадачі.
- Для підзадач розробляється (-ються) `ForkJoinTask` клас (-и)
- Підзадачі (-а) додаються в пул потоків і пул запускається на виконання
- З розв'язків підзадач складається розв'язок задачі

Технологія «крадіжки задачі»

Кожний потік («воркер») пулу потоків типу `ForkJoinPool` може виконувати кілька задач одночасно. Кількість воркерів, за замовчуванням, дорівнює кількості доступних процесорів в обчислювальній системі.

Кожний воркер має двосторонню чергу `ForkJoinTask`-задач. При надходженні задачі ставлять за правилом LIFO.

Пришвидшення відбувається за рахунок технології «крадіжки роботи»: воркер, який має порожню чергу задач, бере на виконання задачу з іншого потоку за правилом FIFO.



Застосування класів ForkJoinFramework

- Задачі створюються як такі, що успадковують клас `ForkJoinTask`
- Виконавці створюються як пул потоків з використанням `ForkJoinPool`, який підтримує інтерфейс `ExecutorService`.

ForkJoinTask:

if (my portion of the work is small enough)

do the work directly

else

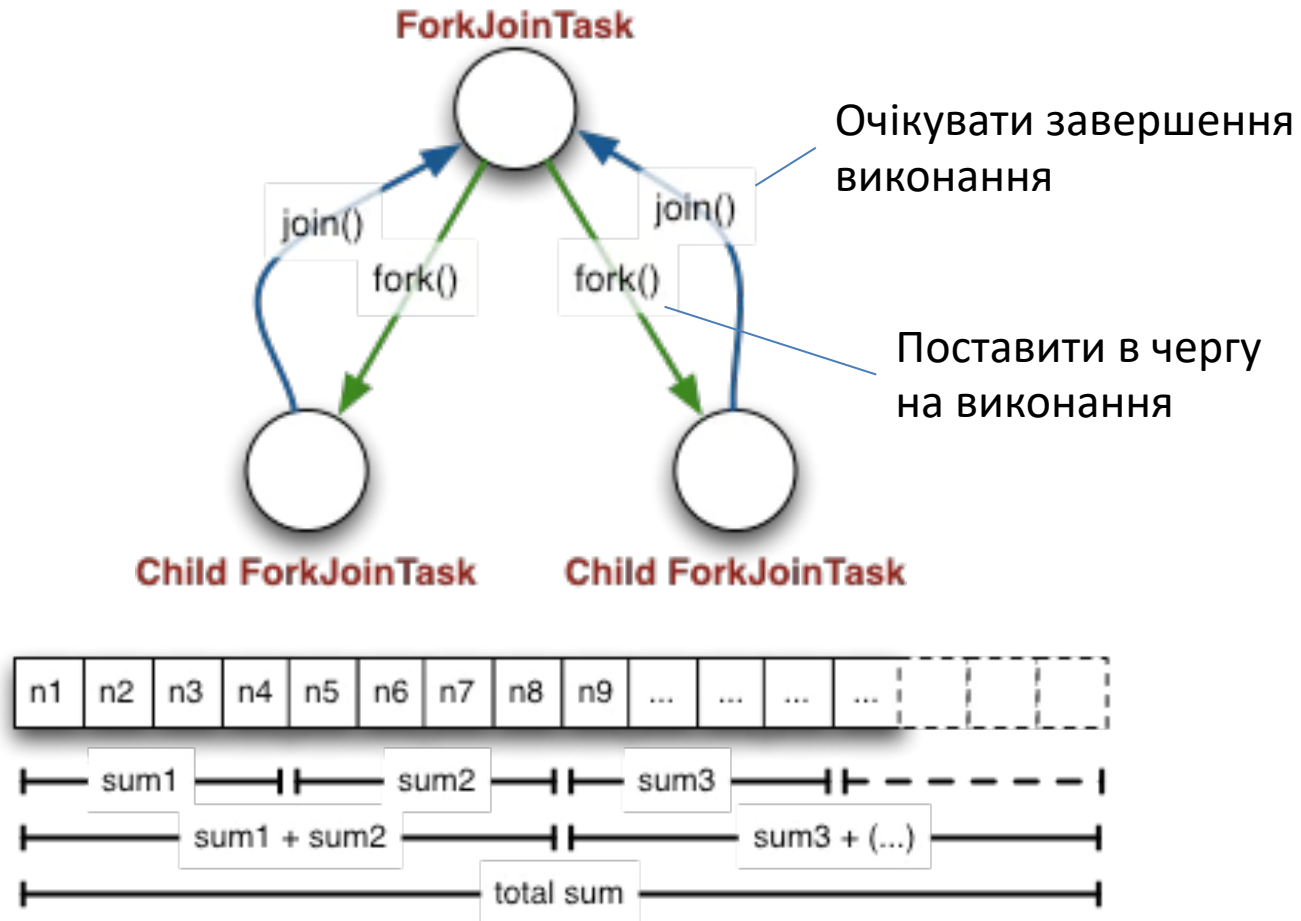
split my work into two pieces

invoke the two pieces and wait for the results

ForkJoinPool:

invoke() all ForkJoinTask objects

Генерування підзадач у ForkJoin Framework



[<https://www.oracle.com/technical-resources/articles/java/fork-join.html>]

Документація класу ForkJoinPool

Class **ForkJoinPool**

`java.lang.Object`

`java.util.concurrent.AbstractExecutorService`

`java.util.concurrent.ForkJoinPool`

Клас імплементує такі інтерейси:

`Executor`, `ExecutorService`

Методи для запуску задач на виконання:

	Викликається ззовні fork/join обчислень (для пулу)	Викликається всередині fork/join обчислень (для задачі)
Запустити асинхронне виконання	<code>execute(ForkJoinTask task)</code>	<code>ForkJoinTask.fork()</code>
Очікувати і отримати результат	<code>invoke(ForkJoinTask task)</code>	<code>ForkJoinTask.invoke()</code>
Запустити на виконання і отримати Future	<code>submit(ForkJoinTask task)</code>	<code>ForkJoinTask.fork()</code> (ForkJoinTasks are Futures)

Основні методи класу ForkJoinPool

`static ForkJoinPool commonPool()` – returns the common pool instance.

`void execute(ForkJoinTask<?> task)` – arranges for (asynchronous) execution of the given task.

`void execute(Runnable task)` – executes the given command at some time in the future.

`<T> T invoke(ForkJoinTask<T> task)` – performs the given task, returning its result upon completion.

`void shutdown()` – possibly initiates an orderly shutdown in which previously submitted tasks are executed, but no new tasks will be accepted.

`List<Runnable> shutdownNow()` – possibly attempts to cancel and/or stop all tasks, and reject all subsequently submitted tasks.

`<T> ForkJoinTask<T> submit(Callable<T> task)` – submits a value-returning task for execution and returns a `Future` representing the pending results of the task.

`<T> ForkJoinTask<T> submit(ForkJoinTask<T> task)` – submits a `ForkJoinTask` for execution.

`<T> ForkJoinTask<T> submit(Runnable task, T result)` – submits a `Runnable` task for execution and returns a `Future` representing that task.

Документація класу ForkJoinTask

java.util.concurrent

Class ForkJoinTask<V>

java.lang.Object

java.util.concurrent.ForkJoinTask<V>

All Implemented Interfaces:

Serializable, Future<V>

Direct Known Subclasses:

CountedCompleter, RecursiveAction, RecursiveTask

```
public abstract class ForkJoinTask<V>
    extends Object
    implements Future<V>, Serializable
```

Основні методи класу ForkJoinTask

`ForkJoinTask<V> fork()` - arranges to asynchronously execute this task in the pool the current task is running in, if applicable, or using the `ForkJoinPool.commonPool()` if not in `ForkJoinPool()`.

`V get()` - waits if necessary for the computation to complete, and then retrieves its result.

`V get(long timeout, TimeUnit unit)` - waits if necessary for at most the given time for the computation to complete, and then retrieves its result, if available.

`V invoke()` - commences performing this task, awaits its completion if necessary, and returns its result, or throws an `(unchecked) RuntimeException` or `Error` if the underlying computation did so.

`static <T extends ForkJoinTask<?>> Collection<T>`

`invokeAll(Collection<T> tasks)` - forks all tasks in the specified collection, returning when `isDone` holds for each task or an `(unchecked)` exception is encountered, in which case the exception is rethrown.

`V join()` - returns the result of the computation when it is done.

Приклад розробки ForkJoinTask-задачі

```
public class ForkBlur extends RecursiveAction {  
    ...  
    protected void computeDirectly() {  
        ... //обчислення для задачі  
    }  
    ...  
    @Override  
    protected void compute() {  
        if (mLength < sThreshold) {  
            computeDirectly();  
            return;  
        }  
        int split = mLength / 2;  
        invokeAll(  
            new ForkBlur(mSource, mStart, split, mDestination),  
            new ForkBlur(mSource, mStart + split,  
                mLength - split, mDestination)  
        );  
    }  
}
```

Структура програми паралельних обчислень в ForkJoin Framework

1. Create a task that represents all of the work to be done.

```
// source image pixels are in src
// destination image pixels are in dst
ForkBlur fb = new ForkBlur(src, 0, src.length, dst);
```

2. Create the `ForkJoinPool` that will run the task.

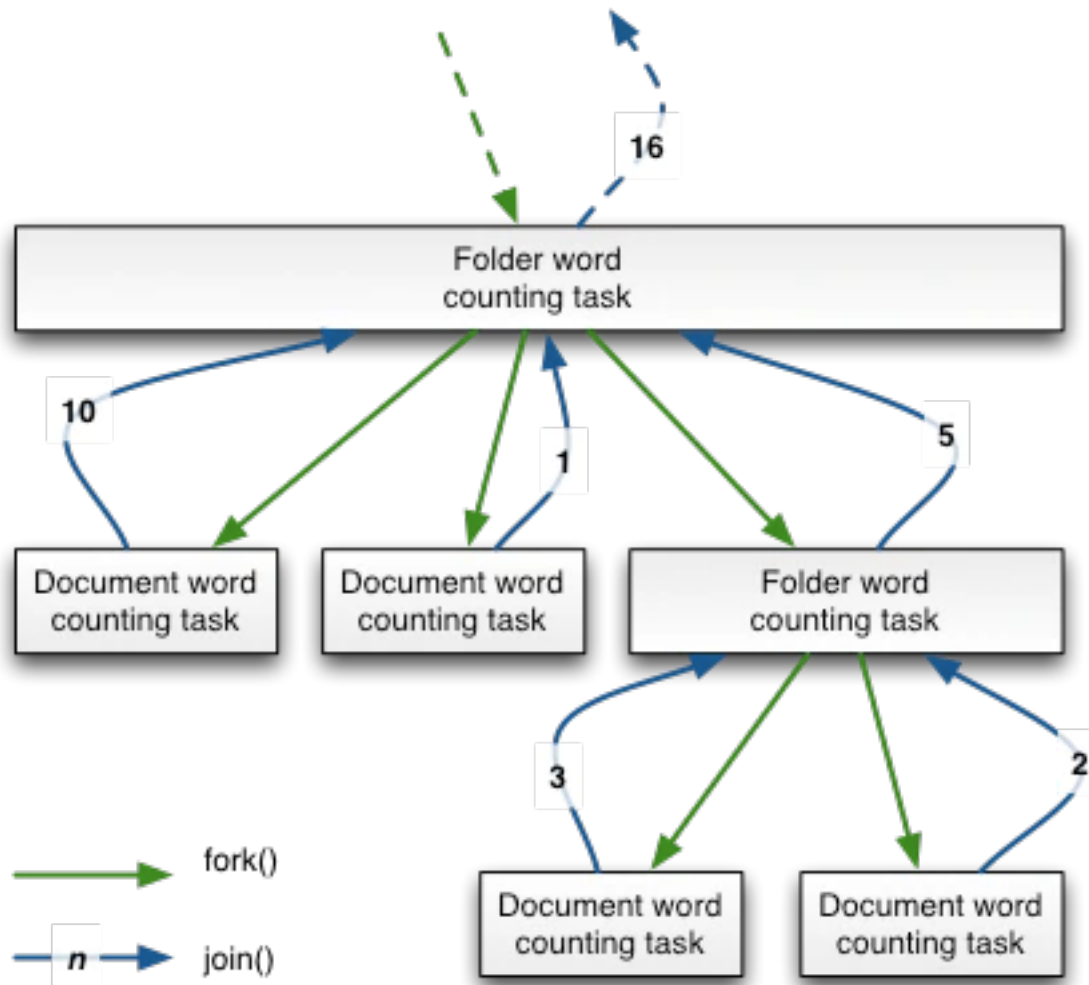
```
ForkJoinPool pool = new ForkJoinPool();
```

3. Run the task.

```
pool.invoke(fb);
```

Приклад: Counter of Searched Word

<https://www.oracle.com/technical-resources/articles/java/fork-join.html>



Приклад: Counter of Searched Word

<https://www.oracle.com/technical-resources/articles/java/fork-join.html>

```
class Document {
    private final List<String> lines;
    ....

    static Document fromFile(File file)
                                throws IOException {
        List<String> lines = new LinkedList<>();
        try(BufferedReader reader =
            new BufferedReader(new FileReader(file))) {
            String line = reader.readLine();
            while (line != null) {
                lines.add(line);
                line = reader.readLine();
            }
        }
        return new Document(lines);
    }
}
```

Приклад: Counter of Searched Word

```
public class Folder {
    private final List<Folder> subFolders;
    private final List<Document> documents;
    ...

    static Folder fromDirectory(File dir) throws IOException {
        List<Document> documents = new LinkedList<>();
        List<Folder> subFolders = new LinkedList<>();

        for (File entry : dir.listFiles()) {
            if (entry.isDirectory()) {
                subFolders.add(Folder.fromDirectory(entry));
            } else {
                documents.add(Document.fromFile(entry));
            }
        }
        return new Folder(subFolders, documents);
    }
}
```


Приклад: Counter of Searched Word

```
public class WordCounter {
    private final ForkJoinPool forkJoinPool =
        new ForkJoinPool();

    String[] wordsIn(String line) {
        return line.trim().split("(\\s|\\p{Punct})+");
    }

    Long occurrencesCount( Document document,
        String searchedWord) {
        long count = 0;
        for (String line : document.getLines()) {
            for (String word : wordsIn(line)) {
                if (searchedWord.equals(word)) {
                    count++;
                }
            }
        }
        return count;
    }
}
```

Приклад: Counter of Searched Word

```
Long countOccurrencesOnSingleThread( Folder folder,
                                     String searchedWord) {
    long count = 0;
    for (Folder subFolder : folder.getSubFolders()) {
        count = count + countOccurrencesOnSingleThread(
            subFolder, searchedWord);
    }
    for (Document document : folder.getDocuments()) {
        count = count + occurrencesCount(document,
            searchedWord);
    }
    return count;
}
```

Приклад: Counter of Searched Word

```
class DocumentSearchTask extends RecursiveTask<Long> {
    .....
}
class FolderSearchTask extends RecursiveTask<Long> {
    .....
}
Long countOccurrencesInParallel(Folder folder,
                                String searchedWord) {
    return forkJoinPool.invoke(
        new FolderSearchTask(folder, searchedWord));
}
}
```

```
class DocumentSearchTask extends RecursiveTask<Long> {
    private final Document document;
    private final String searchedWord;
    .....
    @Override
    protected Long compute() {
        return occurrencesCount(document, searchedWord);
    }
}
```

```
class FolderSearchTask extends RecursiveTask<Long> {
    private final Folder folder;
    private final String searchedWord;

    .....
    @Override
    protected Long compute() {
        long count = 0L;
        List<RecursiveTask<Long>> tasks = new LinkedList<>();

        for (Document document : folder.getDocuments()) {
            DocumentSearchTask task =
                new DocumentSearchTask(document, searchedWord);
            tasks.add(task);
            task.fork();
        }

        for (RecursiveTask<Long> task : tasks) {
            count = count + task.join();
        }
        return count;
    }
}
```

```
public class WordCounter {
    public static void main(String[] args) throws IOException {
        Folder folder =
            Folder.fromDirectory(new File("D:/TextFolder"));
        WordCounter wordCounter = new WordCounter();
        String searchedWord = "synchronized";
        final int repeatCount = 4;
        long cstartTime, stopTime, counts=0, averTime = 0;
        long averTime = 0;
        for (int i = 0; i < repeatCount; i++) {
            startTime = System.currentTimeMillis();
            counts = wordCounter.countOccurrencesInParallel(folder,
                                                            searchedWord);

            stopTime = System.currentTimeMillis();
            averTime+=stopTime - startTime;
        }
        System.out.println(counts +
            " words are found. Fork / join search took " +
            averTime/repeatCount+ "ms");
    }
}
```

```
averTime = 0;
for (int i = 0; i < repeatCount; i++) {
    startTime = System.currentTimeMillis();
    counts = wordCounter.countOccurrencesOnSingleThread(
                                                folder, searchedWord)

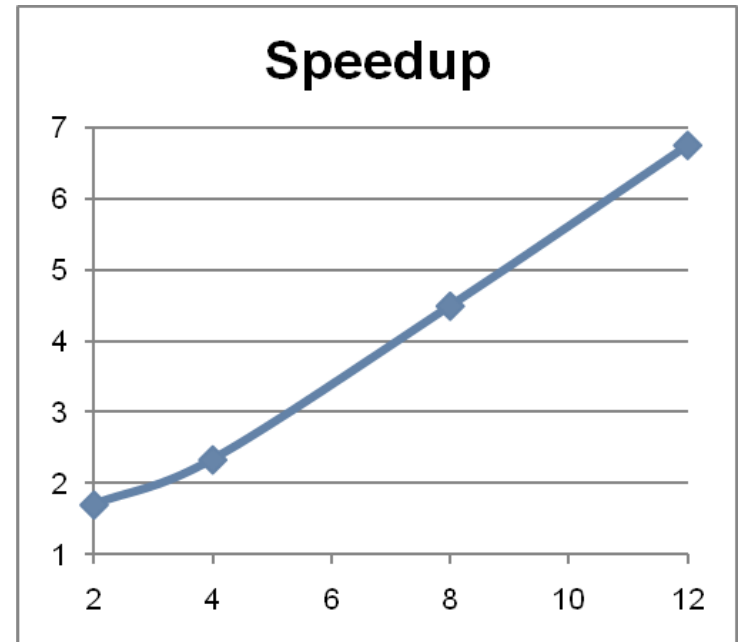
    stopTime = System.currentTimeMillis();
    averTime+=stopTime - startTime;
}
System.out.println(counts +
                    " words are fined. Single thread search took " +
                    averTime/repeatCount+ "ms");
}
}
```

Ефективність ForkJoin Framework

Результат дослідження ефективності алгоритму пошуку слів в документах при запуску на різній кількості ядер Sun Fire T2000 server from Oracle [<https://www.oracle.com/technical-resources/articles/java/fork-join.html>]:

Number of Cores	Single-Thread Execution Time (ms)	Fork/Join Execution* Time (ms)	Speedup
2	18798	11026	1.704879376
4	19473	8329	2.337975747
8	18911	4208	4.494058935
12	19410	2876	6.748956885

*Час виконання алгоритму оцінювався за найменшим значенням серед кількох запусків.



Приклад: результат запуска

```
14
15     long averTime = 0;
16     for (int i = 0; i < repeatCount; i++) {
17         startTime = System.currentTimeMillis();
18         counts = wordCounter.countOccurrencesInParallel(folder, searchedWord);
19         stopTime = System.currentTimeMillis();
20         averTime+=stopTime - startTime;
21     }
22     System.out.println(counts + " words are fined. Fork / join search took " +averTime);
23
24     averTime = 0;
25     for (int i = 0; i < repeatCount; i++) {
```

ForkJoinExample.WordCounter > main > for (int i = 0; i < repeatCount; i++) >

Ввод - Threads (run) ✖

run:

36 words are fined. Fork / join search took 134ms

36 words are fined. Single thread search took 42ms

СБОРКА УСПЕШНО ЗАВЕРШЕНА (общее время: 1 секунда)